Em Java, a estrutura `try-catch-finally` é usada para **tratar exceções** (erros ou eventos inesperados que acontecem durante a execução do programa) e garantir que o sistema não feche de forma abrupta. [1, 2, 3]

O que cada bloco faz

- **`try` (Tentar):** É onde você coloca o código "arriscado", ou seja, os comandos que **podem** gerar uma falha ou exceção (como ler um arquivo que não existe ou dividir um número por zero). [1, 2, 3]
- **`catch` (Capturar):** É o bloco que "pega" a exceção caso ela aconteça dentro do `try`. Ele define qual tipo de erro você está esperando e o que o programa deve fazer para continuar funcionando (por exemplo, exibir uma mensagem amigável ao usuário). [1, 2, 3]
- **`finally` (Finalmente):** É um bloco de execução **obrigatória**. O código que estiver aqui dentro será executado **sempre**, quer uma exceção tenha ocorrido ou não, e mesmo que você use um `return` dentro do `try` ou do `catch`. É muito utilizado para fechar conexões com bancos de dados, arquivos abertos ou liberar recursos. [1, 2, 3, 4]

Exemplo prático

java

```
try {
    // Código que pode dar erro (ex: divisão por zero)
    int resultado = 10 / 0;
    System.out.println("Resultado: " + resultado);
} catch (ArithmeticException e) {
    // Tratamento do erro específico
    System.out.println("Erro: Não é possível dividir por zero!");
} finally {
    // Bloco que roda sempre no final
    System.out.println("Operação finalizada.");
}
```

Use o código com cuidado.

- Para mais detalhes sobre o funcionamento e boas práticas, confira o artigo explicativo na DIO ou a documentação oficial da Oracle. [1, 2]